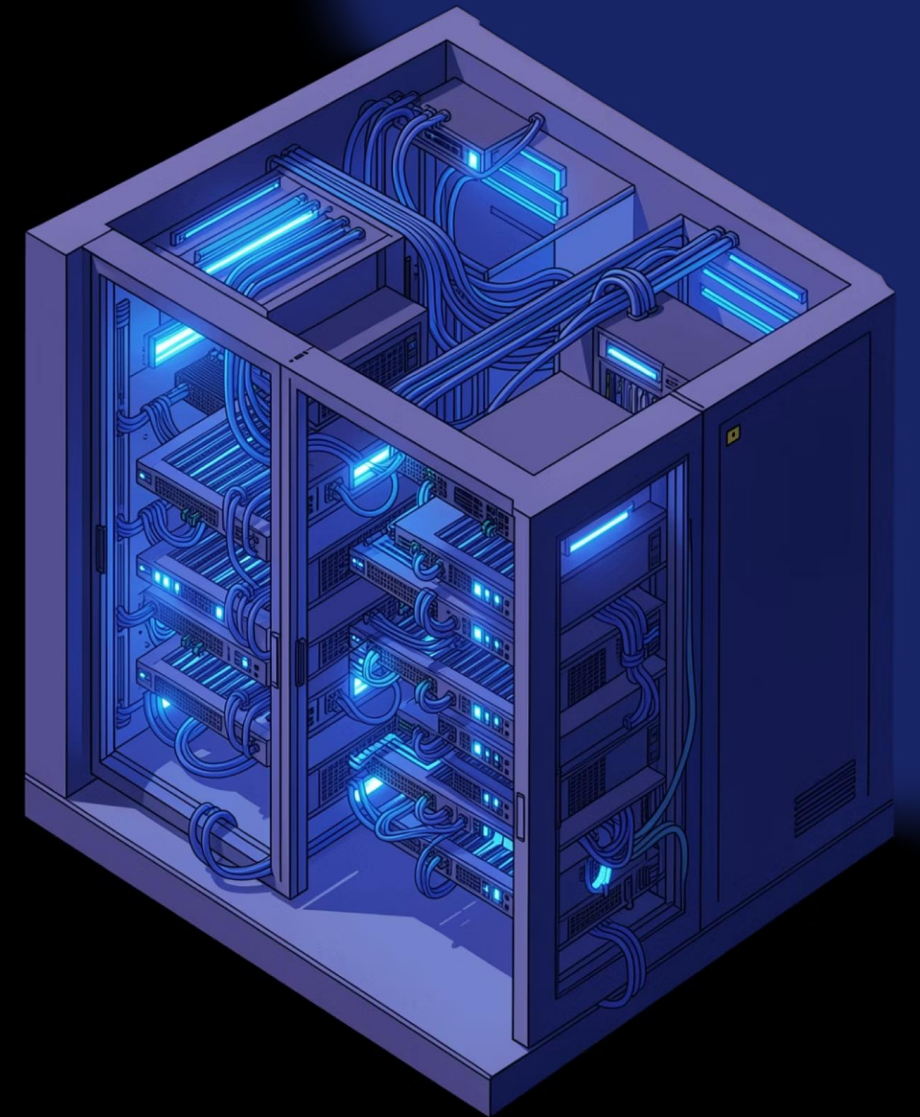


Oracle PL/SQL – Procedure, Function, Package e Trigger

Dal codice singolo al codice riutilizzabile

In questa lezione facciamo un salto di qualità: abbandoniamo il codice PL/SQL scritto "al volo" e iniziamo a costruire **oggetti di programma strutturati, riutilizzabili e professionali**. Partendo dalle conoscenze già acquisite — blocchi anonimi, variabili, cursori, cicli e condizioni — impareremo a organizzare la logica di business in componenti modulari che possono essere richiamati, condivisi e mantenuti nel tempo.

Il problema del codice duplicato è reale: la stessa logica di calcolo scritta in dieci punti diversi diventa un incubo da aggiornare. I subprogrammi PL/SQL risolvono questo problema offrendo un unico punto di definizione e molteplici punti di utilizzo.



Cos'è una PROCEDURE

Una **PROCEDURE** è un blocco PL/SQL con nome, memorizzato direttamente nel dizionario dati di Oracle. A differenza di un blocco anonimo, una procedura viene **compilata una volta sola** e può essere richiamata infinite volte da qualsiasi sessione o blocco PL/SQL.

Le caratteristiche fondamentali di una procedure sono:

Esegue un'azione

Modifica dati, invia messaggi, chiama altre procedure. È pensata per il **lavoro**, non per il calcolo.

Non restituisce un valore

Non può essere usata in una SELECT. Il risultato è l'**effetto collaterale** sull'ambiente (dati modificati, righe inserite).

Può avere parametri

Accetta input tramite parametri formali, rendendo la stessa procedura utilizzabile con dati diversi.

Sintassi e Esempio

```
CREATE OR REPLACE PROCEDURE update_salary(  
    p_id      IN NUMBER,  
    p_factor  IN NUMBER DEFAULT 1.1  
) IS  
BEGIN  
    UPDATE employees  
        SET salary = salary * p_factor  
        WHERE employee_id = p_id;  
    COMMIT;  
END update_salary;  
/
```

`CREATE OR REPLACE` garantisce che la procedura venga ricreata senza errori se esiste già. I parametri con `DEFAULT` sono opzionali alla chiamata.

Come usare una PROCEDURE

Una volta creata, una procedura può essere richiamata da qualsiasi blocco PL/SQL (anonimo, procedure, function, trigger) oppure da strumenti come SQL*Plus, SQL Developer o applicazioni esterne tramite chiamate JDBC/ODBC. La chiamata avviene tramite il nome della procedura seguito dai parametri tra parentesi.

Chiamata da blocco PL/SQL

```
-- Chiamata con parametro posizionale
BEGIN
  update_salary(101);
END;
/

-- Chiamata con parametro nominato
BEGIN
  update_salary(
    p_id      => 101,
    p_factor => 1.15
  );
END;
/

-- Chiamata multipla in loop
BEGIN
  FOR r IN (SELECT employee_id FROM employees
            WHERE department_id = 50) LOOP
    update_salary(r.employee_id, 1.10);
  END LOOP;
END;
/
```

Gestione transazionale: COMMIT e ROLLBACK

Una procedura può contenere istruzioni `COMMIT` e `ROLLBACK`. È importante decidere **chi è responsabile della transazione**:

COMMIT nella procedura

La procedura gestisce autonomamente la persistenza. Utile per operazioni autonome, ma riduce il controllo del chiamante.

COMMIT nel chiamante

Il blocco chiamante decide quando confermare. Permette di raggruppare più operazioni in un'unica transazione atomica.

 **Best Practice:** In applicazioni complesse, evita COMMIT/ROLLBACK dentro le procedure. Lascia la gestione transazionale al livello superiore (chiamante o application layer).

Cos'è una FUNCTION

Sintassi e Esempio

```
CREATE OR REPLACE FUNCTION get_bonus(  
  p_salary IN NUMBER  
) RETURN NUMBER IS  
  v_bonus NUMBER;  
BEGIN  
  v_bonus := p_salary * 0.10;  
  RETURN v_bonus;  
END get_bonus;  
/  
  
-- Uso in SELECT  
SELECT first_name, salary,  
       get_bonus(salary) AS bonus  
FROM employees  
WHERE department_id = 50;
```

Una **FUNCTION** è un subprogramma PL/SQL che **restituisce obbligatoriamente un valore** al chiamante tramite l'istruzione `RETURN`. A differenza delle procedure, le function possono essere usate direttamente nelle query SQL, rendendole strumenti potenti per estendere le capacità del linguaggio SQL.

1 Restituisce un valore

Ogni function deve avere una clausola `RETURN tipo_dato` nell'intestazione e almeno un'istruzione `RETURN` nel corpo. Il tipo restituito può essere `NUMBER`, `VARCHAR2`, `DATE`, `BOOLEAN` (solo in PL/SQL) o un tipo definito dall'utente.

2 Usabile in SQL

Può apparire in `SELECT`, `WHERE`, `ORDER BY`, `GROUP BY` come se fosse una funzione nativa di Oracle. Questo la distingue radicalmente dalle procedure.

3 Nessun effetto collaterale

Per essere usabile in SQL, una function **non dovrebbe modificare dati** (niente DML). Oracle consente di dichiarare restrizioni di purezza con `PRAGMA RESTRICT_REFERENCES`.

Usare una FUNCTION

La function è uno degli strumenti più versatili di PL/SQL. Può essere invocata sia all'interno di query SQL standard sia all'interno di blocchi PL/SQL per assegnare valori a variabili o usarli in espressioni complesse. Vediamo entrambe le modalità con esempi pratici sulla tabella EMPLOYEES.

Uso in SELECT SQL

```
-- Function usata nella lista di selezione
SELECT
  e.first_name,
  e.last_name,
  e.salary,
  pkg_emp.get_bonus(e.salary) AS bonus,
  e.salary + pkg_emp.get_bonus(e.salary)
    AS total_compensation
FROM employees e
WHERE e.department_id = 50
ORDER BY total_compensation DESC;

-- Function usata in WHERE
SELECT first_name, salary
FROM employees
WHERE get_bonus(salary) > 300;
```

La function viene valutata riga per riga. Può essere usata anche in clausole `GROUP BY`, `HAVING` e `ORDER BY`.

Uso in blocco PL/SQL

```
DECLARE
  v_salary  NUMBER := 3500;
  v_bonus   NUMBER;
  v_net     NUMBER;
BEGIN
  -- Assegnazione del valore restituito
  v_bonus := get_bonus(v_salary);
  v_net   := v_salary + v_bonus;

  DBMS_OUTPUT.PUT_LINE(
    'Salario: ' || v_salary ||
    ' | Bonus: ' || v_bonus ||
    ' | Totale: ' || v_net
  );

  -- Uso in espressione
  IF get_bonus(v_salary) > 500 THEN
    DBMS_OUTPUT.PUT_LINE('Bonus elevato!');
  END IF;
END;
/
```

 **Attenzione:** Una function che solleva un'eccezione interromperà la query SQL in cui è usata. Gestire sempre le eccezioni internamente alla function se viene usata in SELECT.

Modalità dei Parametri: IN, OUT, IN OUT

I parametri sono il canale di comunicazione tra il chiamante e il subprogramma. Oracle PL/SQL offre tre modalità di passaggio che controllano direzione e comportamento dei dati. La scelta corretta è fondamentale per la chiarezza e la correttezza del codice.



IN — Input (default)

Il parametro è **solo in lettura** all'interno del subprogramma. Il chiamante passa un valore che non può essere modificato. È la modalità più sicura e la più usata. Esempio: `p_salary IN NUMBER`.



OUT — Output

Il parametro è **solo in scrittura**. Il subprogramma assegna un valore che il chiamante potrà leggere dopo la chiamata. Il valore iniziale è NULL. Utile per restituire più valori da una procedure. Esempio: `p_result OUT NUMBER`.



IN OUT — Entrambi

Il parametro è sia input che output. Il chiamante passa un valore, il subprogramma lo modifica e restituisce il nuovo valore. Usato per trasformazioni in-place. Esempio: `p_string IN OUT VARCHAR2`.

Esempio completo con OUT

```
CREATE OR REPLACE PROCEDURE get_emp_info(
  p_emp_id IN NUMBER,
  p_name OUT VARCHAR2,
  p_salary OUT NUMBER,
  p_status OUT VARCHAR2
) IS
BEGIN
  SELECT first_name || ' ' || last_name,
         salary,
         CASE WHEN commission_pct IS NOT NULL
              THEN 'Con provvigione'
              ELSE 'Fisso' END
  INTO p_name, p_salary, p_status
  FROM employees
  WHERE employee_id = p_emp_id;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    p_name := NULL;
    p_salary := NULL;
    p_status := 'Non trovato';
END get_emp_info;
/

-- Chiamata
DECLARE
  v_name VARCHAR2(100);
  v_sal NUMBER;
  v_status VARCHAR2(50);
BEGIN
  get_emp_info(101, v_name, v_sal, v_status);
  DBMS_OUTPUT.PUT_LINE(v_name || ': ' || v_sal);
END;
/
```

Tabella riassuntiva

Modalità	Direzione	Modificabile?
IN	Chiamante → Subprog.	No
OUT	Subprog. → Chiamante	Sì (solo scrittura)
IN OUT	Bidirezionale	Sì



I parametri IN sono passati **per riferimento** (efficienti). OUT e IN OUT possono richiedere copia del dato.

Organizzare il codice con i PACKAGE

Un **PACKAGE** è un contenitore logico che raggruppa procedure, function, variabili, costanti, tipi e cursori correlati tra loro. È l'unità di modularità più avanzata in PL/SQL e rappresenta la best practice per organizzare codice di produzione in Oracle Database.

Struttura: SPEC e BODY

1	2
PACKAGE SPEC (Specification) È l' interfaccia pubblica del package. Contiene le dichiarazioni (signature) di procedure e function, senza il corpo. Il chiamante vede solo ciò che è nella SPEC. Tutto ciò che non è dichiarato qui è privato .	PACKAGE BODY Contiene l' implementazione completa . Include il codice delle subprogrammi dichiarati nella SPEC più eventuali subprogrammi privati, variabili di stato e sezioni di inizializzazione (BEGIN...END finale).

Esempio: pkg_emp

```
-- SPEC: interfaccia pubblica
CREATE OR REPLACE PACKAGE pkg_emp AS
  -- Costante pubblica
  c_bonus_std CONSTANT NUMBER := 0.10;

  -- Function pubblica
  FUNCTION get_bonus(p_salary NUMBER)
    RETURN NUMBER;

  -- Procedure pubblica
  PROCEDURE raise_salary(
    p_id      IN NUMBER,
    p_factor  IN NUMBER DEFAULT 1.1
  );
END pkg_emp;
/

-- BODY: implementazione
CREATE OR REPLACE PACKAGE BODY pkg_emp AS
  -- Variabile privata (non visibile fuori)
  v_log_count NUMBER := 0;

  FUNCTION get_bonus(p_salary NUMBER)
    RETURN NUMBER IS
  BEGIN
    RETURN p_salary * c_bonus_std;
  END;

  PROCEDURE raise_salary(
    p_id      IN NUMBER,
    p_factor  IN NUMBER DEFAULT 1.1
  ) IS
  BEGIN
    UPDATE employees
      SET salary = salary * p_factor
      WHERE employee_id = p_id;
    v_log_count := v_log_count + 1;
  END;
END pkg_emp;
/
```

- Modularità**
Raggruppa logicamente le funzionalità (es. `pkg_emp`, `pkg_order`, `pkg_report`).
- Stato persistente**
Le variabili di package mantengono il valore per tutta la sessione, tra una chiamata e l'altra.

- Incapsulamento**
Nasconde i dettagli implementativi. Cambia il BODY senza impattare i chiamanti.
- Performance**
I package vengono caricati in memoria una volta sola e rimangono disponibili per tutta la sessione.

Automatizzare con i TRIGGER

Un **TRIGGER** è un blocco PL/SQL memorizzato nel database che viene **eseguito automaticamente** da Oracle in risposta a un evento specifico. A differenza di procedure e function, un trigger non viene mai chiamato esplicitamente: è il motore del database a invocarlo quando si verifica la condizione definita.

Tipi di Trigger DML

→ BEFORE INSERT / UPDATE / DELETE

Eseguito **prima** dell'operazione. Ideale per validazioni, default di valori, trasformazioni dati. Può modificare i dati in ingresso tramite `:NEW`.

→ AFTER INSERT / UPDATE / DELETE

Eseguito **dopo** l'operazione. Ideale per audit trail, propagazione a tabelle collegate, notifiche. I dati sono già stati scritti.

→ FOR EACH ROW (row-level)

Eseguito **una volta per ogni riga** interessata. Ha accesso a `:NEW` e `:OLD`. È il tipo più comune.

→ Statement-level (senza FOR EACH ROW)

Eseguito **una sola volta** per l'intera operazione, indipendentemente dal numero di righe. Non ha accesso a `:NEW`/`:OLD`.

Pseudo-record: :NEW e :OLD

In un trigger row-level, Oracle mette a disposizione due pseudo-record che rappresentano lo stato della riga:

- `:NEW.colonna` — valore **nuovo** (dopo INSERT o UPDATE; NULL in DELETE)
- `:OLD.colonna` — valore **vecchio** (prima di UPDATE o DELETE; NULL in INSERT)

Esempio: Trigger di default salary

```
CREATE OR REPLACE TRIGGER trg_emp_salary
  BEFORE INSERT ON employees
  FOR EACH ROW
  BEGIN
    -- Se salary non specificato, imposta default
    :NEW.salary := NVL(:NEW.salary, 1000);

    -- Se hire_date non specificato, imposta oggi
    IF :NEW.hire_date IS NULL THEN
      :NEW.hire_date := TRUNC(SYSDATE);
    END IF;
  END trg_emp_salary;
/

-- Test: salary non passato
INSERT INTO employees (
  employee_id, first_name, last_name,
  job_id, department_id
) VALUES (
  999, 'Mario', 'Rossi',
  'IT_PROG', 50
);
-- salary sarà automaticamente 1000
```



Trigger di Audit: Un uso classico è tracciare le modifiche in una tabella

Procedure vs Function vs Package vs Trigger

Questa è la slide più importante della lezione. Sapere **quale strumento usare e quando** è la differenza tra scrivere codice PL/SQL e scrivere codice PL/SQL *professionale*. La scelta sbagliata porta a codice difficile da mantenere, testare e ottimizzare.



PROCEDURE

Esegue azioni, modifica dati.
Nessun valore restituito.
Chiamata esplicita.
INSERT, UPDATE, DELETE.



FUNCTION

Calcola e restituisce un valore.
Usabile in SELECT.
Restituisce con RETURN.
No modifiche dati.



PACKAGE

Organizza e raggruppa
procedure e function.
Modularità, incapsulamento.
Separa interfaccia da
implementazione.

SPEC
BODY



TRIGGER

Eseguito automaticamente dal
dal DB per eventi DML.
Usa  :NEW e :OLD.
Mai chiamata esplicitamente.
Legato a tabelle.

Domande guida per la scelta

- Devo modificare dati nel database?
→ Usa una **PROCEDURE**. È lo strumento naturale per operazioni DML.
- Devo calcolare e restituire un valore?
→ Usa una **FUNCTION**. Soprattutto se vuoi usarla in una SELECT.
- Ho più subprogrammi correlati?
→ Raggruppalì in un **PACKAGE**. Migliora organizzazione, riuso e performance.
- Devo reagire automaticamente a un evento?
→ Usa un **TRIGGER**. Non richiede chiamata esplicita, si attiva da solo.

Tabella di riferimento rapido

Oggetto	Restituisce valore?	Usabile in SELECT?	Chiamata esplicita?	Caso d'uso tipico
Procedure	No (solo OUT)	No	Sì	INSERT, UPDATE, DELETE
Function	Sì (RETURN)	Sì	Sì	Calcoli, trasformazioni
Package	Contenitore	Dipende	Sì	Organizzazione modulare
Trigger	No	No	No (automatico)	Audit, validazioni, default

 **Regola d'oro:** Se stai scrivendo un trigger che fa DML su un'altra tabella, valuta se non sia meglio usare una procedure chiamata esplicitamente. I trigger

Cosa abbiamo imparato — Esercizi Pratici

Abbiamo coperto i quattro pilastri della programmazione PL/SQL strutturata. Ora è il momento di consolidare le conoscenze con esercizi pratici. Ogni esercizio costruisce sul precedente, guidandoti verso la creazione di un package completo con trigger di supporto.

01	02	03
Procedure → Azioni Esegue operazioni sul database (DML). Parametri IN/OUT per comunicare con il chiamante.	Function → Valori Calcola e restituisce un valore. Usabile in SELECT e in espressioni PL/SQL.	Parametri → Comunicazione IN, OUT, IN OUT controllano direzione e comportamento del flusso dati.
04	05	
Package → Organizzazione Raggruppa, incapsula e ottimizza subprogrammi correlati con SPEC e BODY.	Trigger → Automazione Reagisce automaticamente a eventi DML con BEFORE/AFTER e :NEW/:OLD.	

🦊 Esercizi proposti

1

Crea una PROCEDURE
Scrivi `raise_all_salaries(p_dept IN NUMBER, p_pct IN NUMBER)` che aumenta del p% lo stipendio di tutti i dipendenti di un dipartimento. Gestisci l'eccezione se il dipartimento non esiste.

2

Crea una FUNCTION
Scrivi `get_annual_salary(p_emp_id IN NUMBER) RETURN NUMBER` che restituisce lo stipendio annuale (`salary × 12 + commission`). Usala in una SELECT su EMPLOYEES.

3

Costruisci un PACKAGE
Crea `pkg_hr` con SPEC e BODY. Inserisci: una function per il bonus, una procedure per l'aggiornamento stipendio, e una variabile privata che conta le operazioni eseguite.

4

Scrivi un TRIGGER
Crea un trigger `BEFORE UPDATE OF salary ON employees` che impedisce una riduzione dello stipendio sollevando `RAISE_APPLICATION_ERROR` se `:NEW.salary < :OLD.salary`.

Prossimi passi

Con questi strumenti hai le basi per scrivere codice PL/SQL professionale. I prossimi argomenti del percorso includono:

- **Gestione avanzata delle eccezioni** — eccezioni definite dall'utente, pragma EXCEPTION_INIT
- **Cursori parametrizzati e REF CURSOR** — per risultati dinamici e flessibili
- **Record e tipi definiti dall'utente** — TYPE, TABLE OF, RECORD
- **Dynamic SQL** — EXECUTE IMMEDIATE per SQL generato a runtime
- **Ottimizzazione e bulk operations** — BULK COLLECT, FORALL

 **Consiglio finale:** Organizza sempre il tuo codice in package fin dal primo giorno. Refactoring successivo è molto più costoso.